

Chia cell code để trình bày

Từ RAG đến GraphRAG: Trợ lý hỏi đáp thị trường bất động sản Việt Nam

Bộ khung 5 notebook, quy ước cell LIVE, CACHE và ẨN, hợp đồng giữa các notebook và kế hoạch trình diễn.

PHẠM VI	1 notebook dùng chung và 4 notebook theo người trình bày, tổng 51 cell
NHÃN CHẠY	19 LIVE · 22 CACHE · 10 ẨN
DATASET	tinixai/vietnam-real-estates (Hugging Face)
ĐI KÈM	Kế hoạch thiết kế slide: 32 slide, 35 phút
TRẠNG THÁI	Khung cell hoàn chỉnh. Code thật và số liệu thật chèn sau

P1-06 · LIVE Chunking: n = 7 khớp với đếm thực	51	cell
P2-06 · LIVE RRF gộp hai bảng xếp hạng	19	LIVE
P3-10 · LIVE Cypher tổng hợp khớp pandas groupby	22	CACHE
P4-10 · CACHE Chạy benchmark ba hệ thống, lưu kết quả	10	ẨN

Mục lục

1 Quy ước chung cho mọi cell	3
1.1 Ba nhãn chạy	3
1.2 Quy ước cell	3
1.3 Giải phẫu một cell	3
1.4 Thư viện gợi ý	4
2 Kiến trúc dự án và hợp đồng giữa các notebook	4
2.1 Cấu trúc thư mục	4
2.2 Luồng dữ liệu giữa các notebook	5
2.3 Hợp đồng giữa các notebook	5
2.4 Bốn notebook trong một bảng	5
2.5 Notebook 00: thiết lập dùng chung	6
3 Người 1: Notebook 1	7
3.1 Notebook 1: Nền tảng RAG và Indexing	7
4 Người 2: Notebook 2	8
4.1 Notebook 2: Retrieval và Generation	8
5 Người 3: Notebook 3	9
5.1 Notebook 3: Knowledge Graph	9
6 Người 4: Notebook 4	10
6.1 Notebook 4: GraphRAG, Hybrid và Đánh giá	10
7 Kế hoạch trình diễn và dự phòng	11
7.1 Tải LIVE theo từng người	11
7.2 Tải LIVE theo từng slide	11
7.3 Phương án dự phòng	11
7.4 Checklist trước ngày trình bày	12
Phụ lục A Mã mẫu các cell kiểm tra (toy)	13
Phụ lục B Tra cứu từ slide sang cell	15

Cách dùng tài liệu này

Mục 1 và 2 để cả nhóm thống nhất quy ước và hợp đồng giữa các notebook. Mục 3 đến 6 là bảng cell của từng người, đọc theo thứ tự chạy. Mục 7 là kế hoạch trình diễn và dự phòng. Phụ lục A có mã mẫu các cell kiểm tra, đã chạy thử và các assert đều đạt. Phụ lục B tra cứu từ slide sang cell.

1 Quy ước chung cho mọi cell

Bộ cell chia thành bốn notebook theo bốn người trình bày, cộng một notebook thiết lập dùng chung. Mọi cell theo cùng một quy ước để trình bày trên lớp không bị chậm và số liệu không bị lệch so với slide.

Tóm tắt

Tổng cộng 51 cell: 19 cell LIVE chạy trước lớp, 22 cell CACHE chạy sẵn và chỉ đọc kết quả, 10 cell ẨN là hàm phụ trợ không hiện. Mỗi người có 4 đến 6 cell LIVE. Chờ chờ số liệu thật đều ghi **ĐIỀN SAU**.

1.1 Ba nhãn chạy

Bảng 1. Ý nghĩa ba nhãn chạy của cell.

NHÃN	NGHĨA	QUY TẮC
LIVE	Chạy trước lớp	Tối đa khoảng 30 giây. Có dự phòng: clip 30 giây và bản HTML đã xuất.
CACHE	Đã chạy sẵn	Trên lớp chỉ đọc kết quả từ đĩa và hiện thống kê. Dùng cho mọi cell nặng.
ẨN	Hàm phụ trợ	Đặt trong src/, không hiện trên lớp. Chỉ in kết quả khi cần kiểm tra.

1.2 Quy ước cell

Bảng 2. Quy ước áp dụng cho mọi notebook.

Đặt tên	P{phần}-{số}, ví dụ P2-06. Notebook thiết lập dùng 00-{số}. Tiêu đề markdown phía trên mỗi cell là câu kết luận của slide tương ứng.
Độ dài	Mỗi cell một ý. Hiện thị tối đa khoảng 15 dòng code và 10 dòng output.
Cell nặng	Embed toàn bộ dữ liệu, nạp đồ thị, chạy benchmark: chạy trước, lưu cache. Trên lớp chỉ load rồi in thống kê.
Cell kiểm tra (toy)	Mỗi công thức có một cell ví dụ nhỏ kèm assert khớp số trên slide. Số đó tính bằng code, không gõ cứng.
Giữ chỗ	Chờ chờ số thật ghi ĐIỀN SAU ngay trong markdown để không sót.
Bí mật	Khoá API đọc từ biến môi trường, không ghi trong notebook hay output.
Dữ liệu cá nhân	Số điện thoại và tên người đăng được ẩn ở P1-04, từ đó mọi cell dùng bản đã ẩn.
Tái lập	Cố định seed và ghi phiên bản thư viện ở 00-01.

1.3 Giải phẫu một cell

Mỗi cell gồm một cell markdown ở trên (mã, nhãn, slide, tiêu đề là kết luận, mục tiêu một dòng) và một cell code ở dưới. Hình dưới là cell P2-06 làm mẫu.

MARKDOWN

P2-06 · Hai bảng xếp hạng gộp thành một bảng RRF
LIVE · slide 12 · Người 2
Mục tiêu: chạy toy X, Y, Z (assert) rồi chạy câu thật.

CODE

```
# P2-06 · LIVE
toy = rrf([["X","Y","Z"], ["Y","Z","X"]])
assert [d for d, _ in toy] == ["Y","X","Z"]
print([(d, round(s, 5)) for d, s in toy])
# câu thật
show(rrf([search_dense(q), search_bm25(q)])[:5])
```

- Mã cell, nhãn và slide nằm ở đầu để người xem định vị ngay.
- Tiêu đề là câu kết luận của slide, không ghi tên kỹ thuật chung chung.
- Mục tiêu một dòng, nói được trong 5 giây.

- Code tối đa khoảng 15 dòng, chạy toy trước.
- assert khớp số trên slide; số tính bằng code.
- Hàm phụ trợ như rrf, show nằm trong src/, không hiện.

Hình 1. Cell mẫu P2-06. Phần code chỉ minh họa cấu trúc; mã chạy được của các cell kiểm tra nằm ở Phụ lục A.

1.4 Thư viện gợi ý

Tên và phiên bản thư viện viết từ trí nhớ. Nhóm kiểm tra cài đặt và tính tương thích trước khi chốt, đặc biệt với ragas và leidenalg vì API có thể đổi theo phiên bản.

Bảng 3. Thư viện gợi ý theo nhóm chức năng.

Nhóm	Thư viện	Dùng ở
Dữ liệu, biểu đồ	datasets, pandas, matplotlib	P1
Embedding, vector DB	sentence-transformers, chromadb hoặc qdrant-client	P1-08 đến P1-11, P2
BM25, tách từ	rank_bm25, underthesea hoặc pyvi	P2-05, P2-06
Kiểm tra, đối chiếu	pydantic, scikit-learn (PCA, cosine đối chiếu)	P1-08, P2-02, P2-03
Đồ thị	neo4j hoặc networkx, pyvis	P3, P4-03, P4-04
Cộng đồng	igraph, leidenalg	P3-12
Đánh giá	ragas	P4-09

2 Kiến trúc dự án và hợp đồng giữa các notebook

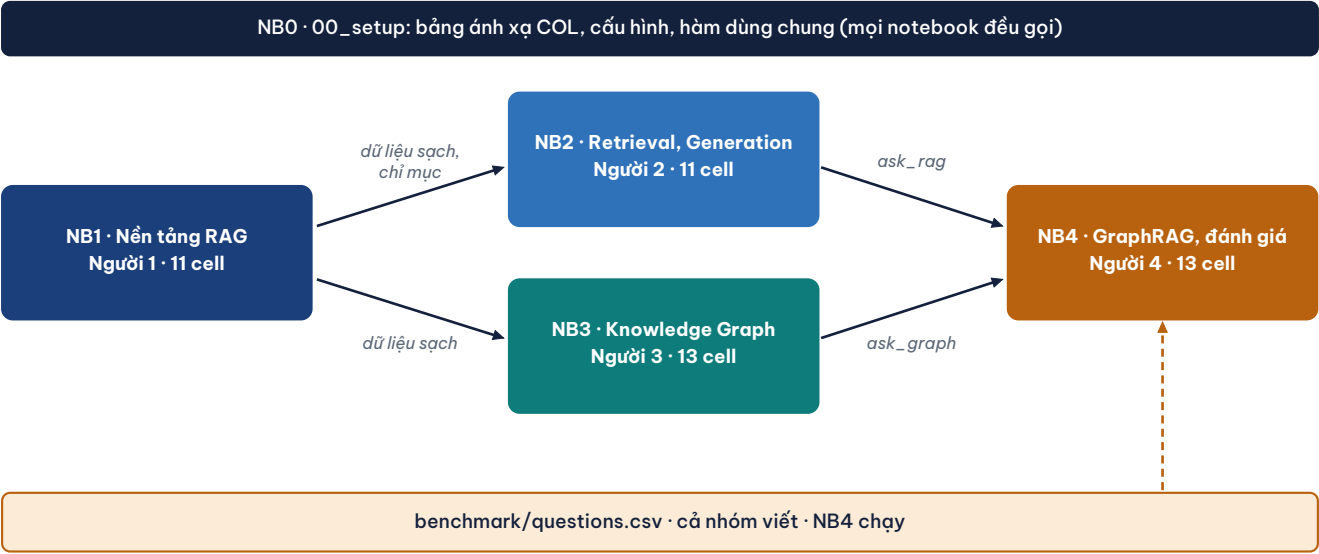
Notebook của người sau chạy trên đầu ra của người trước, nên cả nhóm chốt tên file, tên hàm và cấu trúc thư mục từ mốc M1.

2.1 Cấu trúc thư mục

Bảng 4. Cấu trúc thư mục đề xuất của dự án.

rag-graphrag-bds/	
├── data/	
│ ├── listings_clean.parquet	# NB1 tạo, cả nhóm dùng
├── index/	# vector DB (Chroma hoặc Qdrant)
├── graph/	
│ ├── load.cypher	# script nạp đồ thị
├── benchmark/	
│ ├── questions.csv	# id, type, question, gt_ids, gt_value
├── src/	
│ ├── rag.py	# NB2: search_dense, search_bm25, rrf, ask_rag
│ ├── graph.py	# NB3: run_cypher, ask_graph
│ ├── hybrid.py	# NB4: router, ask_hybrid
│ └── eval.py	# NB4: recall_at_k, mrr, faithfulness
├── notebooks/	
│ ├── 00_setup.ipynb	
│ ├── 01_rag_indexing.ipynb	
│ ├── 02_retrieval_generation.ipynb	
│ ├── 03_knowledge_graph.ipynb	
│ └── 04_graphrag_eval.ipynb	
├── results/	
│ ├── cache/	# kết quả cell CACHE
│ └── results.parquet	# NB4: kết quả benchmark

2.2 Luồng dữ liệu giữa các notebook



Hình 2. Luồng dữ liệu. Mỗi tên là thứ notebook sau cần từ notebook trước; thanh dưới cùng là bộ benchmark dùng chung.

2.3 Hợp đồng giữa các notebook

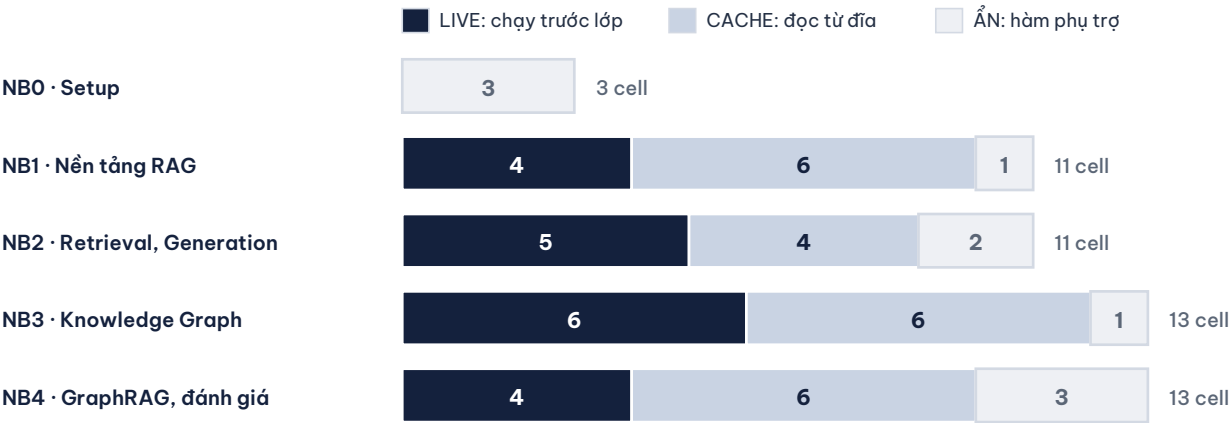
Bảng 5. Những thứ dùng chung cần chốt từ mốc M1.

Thứ dùng chung	Tên đề xuất	Người tạo	Người dùng
Dữ liệu sạch	data/listings_clean.parquet (có listing_id)	1	2, 3, 4
Chỉ mục vector	index/ (Chroma hoặc Qdrant)	1	2, 4
Đồ thị	Neo4j (hoặc graph/graph.gpickle) và script nạp graph/load.cypher	3	4
Bộ benchmark	benchmark/questions.csv: id, type, question, gt_ids, gt_value	Cả nhóm	4
Hàm RAG	src/rag.py: search_dense, search_bm25, rrf, ask_rag	2	4
Hàm đồ thị	src/graph.py: run_cypher, ask_graph	3	4
Hàm hybrid, đánh giá	src/hybrid.py, src/eval.py	4	Cả nhóm

2.4 Bốn notebook trong một bảng

Bảng 6. Tổng quan các notebook.

Notebook	Chủ đề	Người	Slide	Cell	LIVE	CACHE	ẨN
00_setup.ipynb	Thiết lập dùng chung	Người 1	n/a	3	0	0	3
01_rag_indexing.ipynb	Nền tảng RAG và Indexing	Người 1	6–9	11	4	6	1
02_retrieval_generation.ipynb	Retrieval và Generation	Người 2	10–16	11	5	4	2
03_knowledge_graph.ipynb	Knowledge Graph	Người 3	17–23	13	6	6	1
04_graphrag_eval.ipynb	GraphRAG, Hybrid và Đánh giá	Người 4	24–30	13	4	6	3
Tổng				51	19	22	10



Hình 3. Số cell theo nhãn của từng notebook.

2.5 Notebook 00: thiết lập dùng chung

Một nơi duy nhất chứa cấu hình, bảng ánh xạ tên cột và hàm dùng chung, để đổi tên cột hay đổi mô hình chỉ sửa một chỗ. File: notebooks/00_setup.ipynb. Đầu ra: COL (bảng ánh xạ cột), cấu hình LLM, embedding, DB, thư mục src/.

Bảng 7. Bảng cell của notebook 00. Cả ba cell đều ẨN nên không hiện trên lớp.

CELL	NHÃN	VIỆC LÀM	KẾT QUẢ HIỆN RA	SLIDE
00-01	ẨN	Import, seed, đường dẫn, bảng ánh xạ COL (tên cột thật của dataset)	Không hiện	n/a
00-02	ẨN	Cấu hình LLM, embedding, DB (khóa API đọc từ biến môi trường, không ghi trong notebook)	Không hiện	n/a
00-03	ẨN	src/: hàm dùng chung (xem hợp đồng ở mục 2.3)	Không hiện	n/a

3 Người 1: Notebook 1

Người 1 dựng nền: dữ liệu sạch và chỉ mục vector. Mọi người sau đều dùng lại hai thứ này.

3.1 Notebook 1: Nền tảng RAG và Indexing

NOTEBOOK P1 · NGƯỜI 1		Slide 6 – 9
Nền tảng RAG và Indexing		11 cell · 4 LIVE
MỤC TIÊU	Từ tập dữ liệu thô đến chỉ mục vector có thể tìm kiếm: khảo sát, làm sạch, chunk, embed, lưu.	
FILE	notebooks/01_rag_indexing.ipynb	
ĐẦU RA CHO NGƯỜI SAU	data/listings_clean.parquet (có listing_id), thư mục index/.	
TẢI LIVE ƯỚC TÍNH	4 cell LIVE, khoảng 2,0 phút (giả định 30 giây mỗi cell, đo lại khi chạy thật)	

Bảng 8. Bảng cell của notebook P1.

CELL	NHÃN	VIỆC LÀM	KẾT QUẢ HIỆN RA	SLIDE
P1-01	CACHE	Nạp dataset tinixai/vietnam-real-estates	shape, danh sách cột, 3 dòng mẫu ĐIỂN SAU	6
P1-02	CACHE	Khảo sát chất lượng: kiểu dữ liệu, tỷ lệ thiếu, tin trùng, độ dài mô tả	Bảng tóm tắt ĐIỂN SAU	6
P1-03	LIVE	Vẽ phân phối giá/m² và số tin theo tỉnh	2 biểu đồ, tiêu đề là kết luận	6
P1-04	CACHE	Làm sạch: chuẩn hoá đơn vị giá, bỏ trùng, chuẩn hoá địa danh, ẩn số điện thoại	Số dòng trước và sau, ví dụ 1 dòng trước và sau	6
P1-05	ẨN	Lưu listings_clean.parquet có listing_id ổn định (cả nhóm dùng chung)	Tên file, số dòng	6
P1-06	LIVE	Chunking: cắt 1 mô tả dài bằng chiến lược cố định rồi bằng 1 tin = 1 chunk; tính n theo công thức và đối chiếu với số chunk thực tế	n = 7 khớp với đếm thực (L = 2000, S = 400, O = 100)	7
P1-07	CACHE	Dựng chunk cho toàn bộ tin, gán metadata (tỉnh, quận, giá, diện tích, loại hình)	Số chunk, histogram độ dài token ĐIỂN SAU	7
P1-08	LIVE	Embed 5 câu mẫu (2 cặp gần nghĩa, 1 câu khác chủ đề); chiếu 2D bằng PCA	Hình điểm, cặp gần nghĩa nằm gần nhau	8
P1-09	CACHE	Embed toàn bộ chunk, lưu .npy	Thời gian chạy, kích thước vector, dung lượng ĐIỂN SAU	9
P1-10	CACHE	Nạp vào vector DB kèm metadata	count() khớp số chunk	9
P1-11	LIVE	Kiểm tra nhanh: 1 câu hỏi, in top-3	Bảng ID, điểm, giá, địa chỉ	9

Cell cầu nối sang Người 2
P1-11 in top-3 của đúng câu "Căn hộ 2PN ở Thủ Đức giá bao nhiêu?". Câu chuyển: có chỉ mục rồi, Người 2 làm cho nó tìm đúng và trả lời có căn cứ.

4 Người 2: Notebook 2

Người 2 biến chỉ mục thành hệ hỏi đáp: hiểu câu hỏi, tìm bằng dense, BM25 và hybrid, rerank, rồi sinh câu trả lời bám nguồn.

4.1 Notebook 2: Retrieval và Generation

NOTEBOOK P2 · NGƯỜI 2		Slide 10–16
Retrieval và Generation		11 cell · 5 LIVE
MỤC TIÊU	Biến câu hỏi thành truy vấn có bộ lọc, tìm bằng dense, BM25 và hybrid, rerank, rồi sinh câu trả lời có trích dẫn.	
FILE	notebooks/02_retrieval_generation.ipynb	
ĐẦU RA CHO NGƯỜI SAU	src/rag.py: search_dense, search_bm25, rrf, ask_rag.	
TẢI LIVE ƯỚC TÍNH	5 cell LIVE, khoảng 2,5 phút (giả định 30 giây mỗi cell, đo lại khi chạy thật)	

Bảng 9. Bảng cell của notebook P2.

CELL	NHÃN	VIỆC LÀM	KẾT QUẢ HIỆN RA	SLIDE
P2-01	ẨN	Nạp chỉ mục, mô hình embedding từ cache	Không hiện	10
P2-02	CACHE	Hiểu câu hỏi: LLM trích bộ lọc thành JSON, kiểm tra bằng pydantic, đổi 5 tỷ thành 5.000 triệu	Câu hỏi tự nhiên và dict bộ lọc cạnh nhau	10
P2-03	LIVE	Cosine tính tay bằng numpy, đối chiếu với sklearn	1,00 / 0,96 / 0,00 cho q = (3, 4)	11
P2-04	CACHE	Dense search top-k kèm ngưỡng; thử k = 3, 5, 8	Bảng ID, điểm, giá	11
P2-05	CACHE	BM25 với bộ tách từ tiếng Việt; so sánh hai câu: "sổ hồng 2PN" và "gần trường học"	Hai bảng top-5: BM25 thắng câu đầu, dense thắng câu sau ĐIỀN SAU	12
P2-06	LIVE	Hàm rrf. Chạy toy X, Y, Z trước (assert), sau đó chạy câu thật	0,03252 / 0,03227 / 0,03200; bảng thứ hạng dense, BM25, RRF	12
P2-07	LIVE	Pre-filter và post-filter trên cùng một câu hỏi có điều kiện cứng	Số kết quả còn lại của mỗi cách	13
P2-08	CACHE	Rerank cross-encoder: top-50 xuống top-5. Cell tùy chọn, bỏ nếu chậm	Thứ hạng trước và sau	13
P2-09	LIVE	Ghép prompt (chỉ thị, ngữ cảnh, câu hỏi) rồi sinh câu trả lời kèm [Tin#ID]	Prompt rút gọn và câu trả lời có trích dẫn	14
P2-10	ẨN	Đóng gói ask_rag(question) trả về câu trả lời và nguồn	Không hiện	14
P2-11	LIVE	3 câu hỏi: tra cứu (đúng), có ràng buộc (đúng), tổng hợp (sai có chủ đích). Cạnh câu 3 in đáp án pandas groupby	2 kết quả đúng, 1 kết quả sai cạnh đáp án chuẩn ĐIỀN SAU	15–16

Cell cầu nối sang Người 3

P2-11 câu 3: đáp án đúng từ pandas groupby nằm ngay cạnh nhưng RAG không tính ra. Câu chuyển: cần cấu trúc quan hệ và khả năng tính trên toàn tập.

Cell tùy chọn

P2-08 (rerank) bỏ được nếu máy chậm; slide 13 vẫn nói được bằng sơ đồ phễu.

5 Người 3: Notebook 3

Người 3 dựng đồ thị từ dữ liệu sạch của Người 1, kiểm tra nhất quán bằng pandas, rồi truy vấn bằng Cypher.

5.1 Notebook 3: Knowledge Graph

NOTEBOOK P3 · NGƯỜI 3		Slide 17–23 13 cell · 6 LIVE
MỤC TIÊU	Dựng đồ thị bất động sản theo schema, nạp dữ liệu, kiểm tra nhất quán, truy vấn bằng Cypher và chạy thuật toán đồ thị.	
FILE	notebooks/03_knowledge_graph.ipynb	
ĐẦU RA CHO NGƯỜI SAU	Đồ thị Neo4j (hoặc graph/graph.gpickle), graph/load.cypher, src/graph.py: run_cypher, ask_graph.	
TẢI LIVE ƯỚC TÍNH	6 cell LIVE, khoảng 3,0 phút (giả định 30 giây mỗi cell, đo lại khi chạy thật)	

Bảng 10. Bảng cell của notebook P3.

CELL	NHÃN	VIỆC LÀM	KẾT QUẢ HIỆN RA	SLIDE
P3-01	ẨN	Kết nối Neo4j (hoặc NetworkX nếu quy mô nhỏ)	Không hiện	17
P3-02	LIVE	Đồ thị mini 6 nút dựng tay từ 1 tin thật; in danh sách triple	Hình đồ thị và 5–6 triple	17
P3-03	CACHE	Tạo ràng buộc và chỉ mục theo schema (Listing, PropertyType, Ward, District, Province)	Hình schema	19
P3-04	LIVE	Entity resolution: hàm chuẩn hoá địa danh, khoá Ward gồm cả quận và tỉnh	Bảng "trước và sau": "Q.7" thành "Quận 7" ĐIỂN SAU	20
P3-05	CACHE	Nạp node (Province, District, Ward, PropertyType) bằng UNWIND theo lô	Số node mỗi loại	20
P3-06	CACHE	Nạp Listing và các quan hệ	Số node, số edge ĐIỂN SAU	20, 23
P3-07	CACHE	Kiểm tra nhất quán: số node, edge trong đồ thị so với pandas	assert khớp	23
P3-08	CACHE	Tuỳ chọn, chỉ làm nếu dataset có mô tả dài: LLM trích tiện ích trên 20 tin mẫu, kiểm tra tay	Bảng triple trích được và tỷ lệ đúng ĐIỂN SAU	20
P3-09	LIVE	Cypher 1, tra cứu: căn hộ 2PN dưới mức giá cho trước	Bảng kết quả, cạnh SQL tương đương	21
P3-10	LIVE	Cypher 2, tổng hợp: quận có giá/m ² trung bình thấp nhất; assert bằng kết quả pandas groupby	Tên quận và giá trị, hai cách khớp nhau	21
P3-11	LIVE	Cypher 3, đa chặng: Tin, Phường, Quận, Tỉnh, đếm tin theo quận	Top 5 quận theo số tin	21
P3-12	LIVE	Modularity: hai tam giác nối một cạnh, tính Q và assert bằng 5/14. Sau đó dựng đồ thị tương đồng giữa các phường (kNN theo hồ sơ giá và loại hình), chạy Leiden	Q = 0,357; bảng cộng đồng phường kèm giá/m ² trung bình ĐIỂN SAU	22
P3-13	CACHE	Trực quan hoá một quận (tối đa 50 nút) bằng pyvis hoặc ảnh chụp Neo4j	Hình đồ thị	23

Lưu ý cho Người 3

P3-12: Leiden chạy trên đồ thị tương đồng giữa các phường, **không** chạy trên cây hành chính, vì cây không có cạnh chéo nên không có cụm có nghĩa. Nếu dataset không đủ trường để dựng hồ sơ phường, rút P3-12 về phần toy (Q = 0,357) và centrality. **P3-10:** Người 4 dùng đúng truy vấn này làm mẫu cho Text2Cypher ở P4-03. **P3-08:** chỉ làm nếu dataset có cột mô tả dài; nếu không, rút gọn phần trích entity bằng LLM trên slide 20.

6 Người 4: Notebook 4

Người 4 ghép hai nguồn của Người 2 và Người 3, đo bằng bộ benchmark chung, và giữ bảng số liệu cuối cho cả nhóm.

6.1 Notebook 4: GraphRAG, Hybrid và Đánh giá

NOTEBOOK P4 · NGƯỜI 4		Slide 24–30 13 cell · 4 LIVE
MỤC TIÊU	Ghép Basic RAG với đồ thị bằng Text2Cypher và router, đo bằng Recall@k, MRR, RAGAS, rồi so ba hệ thống trên cùng bộ benchmark.	
FILE	notebooks/04_graphrag_eval.ipynb	
ĐẦU RA CHO NGƯỜI SAU	results/results.parquet, src/hybrid.py (router, ask_hybrid), src/eval.py.	
TÀI LIVE ƯỚC TÍNH	4 cell LIVE, khoảng 2,0 phút (giả định 30 giây mỗi cell, đo lại khi chạy thật)	

Bảng 11. Bảng cell của notebook P4.

CELL	NHÃN	VIỆC LÀM	KẾT QUẢ HIỆN RA	SLIDE
P4-01	ẨN	Nạp benchmark/questions.csv, hàm ask_rag của Người 2, kết nối đồ thị của Người 3	Không hiện	24
P4-02	CACHE	Tính đáp án chuẩn bằng pandas groupby cho nhóm câu tổng hợp và đa chọn, ghi vào cột gt_value	3–4 dòng mẫu	28
P4-03	LIVE	Text2Cypher: prompt chứa schema, sinh Cypher, bộ chặn chỉ đọc (từ chối CREATE, DELETE, MERGE, SET), thực thi, thử lại khi lỗi	Cypher sinh ra, kết quả, và 1 ví dụ bị chặn	25
P4-04	CACHE	Local retrieval: mở rộng láng giềng quanh 1 thực thể	Hình đồ thị con	25
P4-05	ẨN	Global (tóm tắt cộng đồng): chỉ giải thích khái niệm, không chạy vì dữ liệu có cấu trúc	Không hiện	25
P4-06	LIVE	Router: phân loại câu hỏi thành basic_rag, graph, hybrid bằng vài ví dụ mẫu; thử trên 8 câu	Bảng câu hỏi, nhãn dự đoán, nhãn đúng	26
P4-07	ẨN	ask_hybrid: gộp ngữ cảnh từ hai nguồn, có đường lui về Basic RAG	Không hiện	26
P4-08	LIVE	Hàm recall_at_k và mrr. Chạy toy trước, sau đó chạy trên benchmark	Recall@5 = 0,67 và MRR = 0,61 ở toy; số thật ĐIỀN SAU	27
P4-09	CACHE	RAGAS (Faithfulness, Answer Relevancy, Context Precision) trên một tập con; kèm hàm Faithfulness thủ công cho toy	Toy 3/4 = 0,75; bảng RAGAS ĐIỀN SAU	28
P4-10	CACHE	Chạy toàn bộ benchmark qua 3 hệ thống, ghi câu trả lời, độ trễ, số token vào results.parquet	Số dòng, tổng thời gian	29–30
P4-11	LIVE	Bảng độ chính xác theo loại câu hỏi và hệ thống; biểu đồ cột nhóm	Bảng và biểu đồ ĐIỀN SAU	30
P4-12	CACHE	Bảng chi phí và độ trễ trung bình theo hệ thống	Bảng ĐIỀN SAU	29
P4-13	CACHE	Phân tích lỗi: 2–3 trường hợp sai (router nhầm, Cypher sai)	Bảng trường hợp và nguyên nhân	30

Lưu ý cho Người 4

P4-03: bộ chặn chỉ đọc bằng regex chỉ là lớp đầu. Nên thêm lớp bảo vệ ở phía cơ sở dữ liệu; nhóm kiểm tra bản Neo4j đang dùng có hỗ trợ phân quyền chỉ đọc không. Mã mẫu ở Phụ lục A. **P4-13:** nếu Hybrid không thắng ở loại câu hỏi nào, giữ nguyên kết quả và giải thích. Kết luận trên slide 30 phải đến từ số đo.

7 Kế hoạch trình diễn và dự phòng

Cell LIVE tốn thời gian của slide chứa nó. Phần này đo tải đó và đề xuất điều chỉnh để không vượt thời lượng.

7.1 Tải LIVE theo từng người

Bảng 12. Tải LIVE so với thời lượng phần của mỗi người. Giả định 30 giây mỗi cell LIVE, đo lại khi chạy thật.

Người	Notebook	Cell LIVE	LIVE (phút)	Thời lượng phần (phút)	Tỷ lệ
Người 1	P1	4	2,0	7,0	29%
Người 2	P2	5	2,5	8,0	31%
Người 3	P3	6	3,0	8,0	38%
Người 4	P4	4	2,0	8,0	25%

Tỷ lệ nằm trong khoảng 25% đến 38% thời lượng phần, nên tổng tải chấp nhận được. Vấn đề nằm ở từng slide cụ thể (mục 7.2).

7.2 Tải LIVE theo từng slide

Bảng 13. Slide có cell LIVE: Ổn nếu tải LIVE không quá 50% thời lượng slide, sát nếu đến 100%, vượt nếu trên 100%.

Slide	Cell LIVE	LIVE (giây)	Slide (giây)	Tỷ lệ	Đánh giá
6	P1-03	30	60	50%	Ổn
7	P1-06	30	90	33%	Ổn
8	P1-08	30	72	42%	Ổn
9	P1-11	30	78	38%	Ổn
11	P2-03	30	60	50%	Ổn
12	P2-06	30	72	42%	Ổn
13	P2-07	30	60	50%	Ổn
14	P2-09	30	60	50%	Ổn
15–16	P2-11	30	168	18%	Ổn
17	P3-02	30	60	50%	Ổn
20	P3-04	30	72	42%	Ổn
21	P3-09, P3-10, P3-11	90	72	125%	Vượt
22	P3-12	30	60	50%	Ổn
25	P4-03	30	72	42%	Ổn
26	P4-06	30	72	42%	Ổn
27	P4-08	30	60	50%	Ổn
30	P4-11	30	96	31%	Ổn

Điều chỉnh đề xuất cho slide 21 và 23

Slide 21 có 3 cell LIVE (P3-09, P3-10, P3-11) tức khoảng 90 giây, trong khi slide chỉ dài 72 giây. Slide 23 (Thực hành 3, 84 giây) hiện chưa có cell LIVE nào. Đề xuất: giữ P3-09 ở slide 21, chuyển P3-10 và P3-11 sang slide 23. Sau điều chỉnh, slide 21 còn 30 giây (42%) và slide 23 có 60 giây (71%).

7.3 Phương án dự phòng

- **Clip dự phòng** 30 giây cho mỗi nhóm cell LIVE. Nếu API hoặc kết nối lỗi thì phát clip.
- **Xuất bản HTML** của mỗi notebook sau khi chạy đủ, để mở khi máy trình chiếu không chạy được code.
- **Cache đầy đủ** trước ngày trình bày; trên lớp cell CACHE chỉ đọc file, không gọi mô hình hay DB nặng.

- **Thử trên máy trình chiếu thật:** kết nối mạng, font tiếng Việt trong output, cỡ chữ code đủ lớn (zoom trình duyệt 125% đến 150%).

7.4 Checklist trước ngày trình bày

☐ Mọi cell CACHE đã chạy, file kết quả có trong results/cache/ và index/	☐ Số trên slide khớp output notebook
☐ Restart & Run ALL thành công trên máy sạch	☐ Mỗi nhóm cell LIVE có clip dự phòng
☐ Không có khoá API trong notebook hay output	☐ Đã xuất bản HTML của 4 notebook
☐ Output không còn số điện thoại hay tên người đăng	☐ Đã thử trên máy trình chiếu thật
☐ Mọi assert của cell toy đều đạt	☐ Mỗi người chạy được notebook của người trước (đúng tên file, tên hàm)
☐ Không còn nhãn ĐIỂN SAU trong markdown	☐ Seed cố định, phiên bản thư viện ghi ở 00-01

Phụ lục A Mã mẫu các cell kiểm tra (toy)

Sáu đoạn mã dưới đây là phần lõi của các cell toy. Cả sáu đã được chạy thử khi dựng tài liệu, mọi assert đều đạt và phần Output là kết quả chạy thật. Số khớp với slide: $n = 7$, cosine 1,00 / 0,96 / 0,00, RRF $Y > X > Z$, $Q = 5/14$, Recall@5 = 0,67, MRR = 0,61, Faithfulness = 0,75. Các đoạn này không gọi dataset, mô hình hay cơ sở dữ liệu, nên chạy được ngay.

A.1 Đếm chunk theo công thức và theo cách cắt thực tế

P1-06 LIVE · slide 7

Kiểm tra công thức $n = \lceil (L - O) / (S - O) \rceil$ bằng cách cắt thật trên chuỗi 2000 token.

```
import math

def chunk_starts(L, S, O):
    """Vị trí bắt đầu của từng chunk khi cắt L token, cỡ S, chồng lấn O."""
    starts, s = [0], 0
    while s + S < L:
        s += S - O
        starts.append(s)
    return starts

L, S, O = 2000, 400, 100
n_thuc = len(chunk_starts(L, S, O))
n_cong_thuc = math.ceil((L - O) / (S - O))
assert n_thuc == n_cong_thuc == 7
print("so chunk:", n_thuc, "| vị trí:", chunk_starts(L, S, O))

# Output
so chunk: 7 | vị trí: [0, 300, 600, 900, 1200, 1500, 1800]
```

A.2 Cosine tính tay và đối chiếu ba vector

P2-03 LIVE · slide 11

Độ gần về hướng không phụ thuộc độ dài: (6, 8) dài gấp đôi q nhưng vẫn cho điểm 1.

```
import numpy as np

def cosine(a, b):
    a, b = np.asarray(a, float), np.asarray(b, float)
    return float(a @ b / (np.linalg.norm(a) * np.linalg.norm(b)))

q = (3, 4)
for d in [(6, 8), (4, 3), (-4, 3)]:
    print(d, f"{cosine(q, d):.2f}")
assert round(cosine(q, (6, 8)), 2) == 1.00
assert round(cosine(q, (4, 3)), 2) == 0.96
assert round(cosine(q, (-4, 3)), 2) == 0.00

# Output
(6, 8) 1.00
(4, 3) 0.96
(-4, 3) 0.00
```

A.3 RRF gộp hai bảng xếp hạng

P2-06 LIVE · slide 12

Toy X, Y, Z: Y thắng dù không đứng đầu bảng nào. Hàm này chuyển vào src/rag.py làm cell ẨN.

```
def rrf(rankings, k=60):
    """rankings: danh sách các bảng xếp hạng, mỗi bảng là list ID theo thứ tự tốt dần."""
    scores = {}
    for ranking in rankings:
        for pos, doc in enumerate(ranking, start=1):
            scores[doc] = scores.get(doc, 0) + 1 / (k + pos)
    return sorted(scores.items(), key=lambda x: -x[1])

dense = ["X", "Y", "Z"]    # hạng theo dense
bm25 = ["Y", "Z", "X"]    # hạng theo BM25
toy = rrf([dense, bm25])
assert [d for d, _ in toy] == ["Y", "X", "Z"]
print([(d, f"{s:.5f}") for d, s in toy])
```

```
# Output
[('Y', '0.03252'), ('X', '0.03227'), ('Z', '0.03200')]
```

A.4 Modularity và degree centrality trên đồ thị toy

P3-12 LIVE · slide 22

Hai tam giác nối bởi một cạnh. Dùng dạng theo cụm: $Q = \sum [L_c/m - (d_c/2m)^2]$.

```
edges = [(0, 1), (1, 2), (0, 2), (3, 4), (4, 5), (3, 5), (2, 3)]
comms = [{0, 1, 2}, {3, 4, 5}]

def modularity(edges, comms):
    m, Q = len(edges), 0.0
    for c in comms:
        L_c = sum(1 for u, v in edges if u in c and v in c)    # cạnh nội bộ cụm
        d_c = sum((u in c) + (v in c) for u, v in edges)    # tổng bậc trong cụm
        Q += L_c / m - (d_c / (2 * m)) ** 2
    return Q

Q = modularity(edges, comms)
assert abs(Q - 5 / 14) < 1e-9
n = 6
deg = {v: sum(v in e for e in edges) for v in range(n)}
centrality = {v: d / (n - 1) for v, d in deg.items()}
assert centrality[2] == centrality[3] == 0.6 and centrality[0] == 0.4
print("Q =", round(Q, 3), "| centrality:", centrality)
```

```
# Output
Q = 0.357 | centrality: {0: 0.4, 1: 0.4, 2: 0.6, 3: 0.6, 4: 0.4, 5: 0.4}
```

A.5 Bộ chặn chỉ đọc cho Text2Cypher (lớp đầu)

P4-03 LIVE · slide 25

Regex chỉ là lớp bảo vệ đầu tiên. Nên thêm lớp ở phía cơ sở dữ liệu; nhóm kiểm tra bản Neo4j đang dùng có hỗ trợ phân quyền chỉ đọc không.

```
import re

FORBIDDEN = re.compile(
    r"\b(CREATE|MERGE|DELETE|DETACH|SET|REMOVE|DROP|LOAD\s+CSV|CALL)\b", re.I)

def is_read_only(cypher: str) -> bool:
    return FORBIDDEN.search(cypher) is None

ok = "MATCH (l:Listing) RETURN count(l) AS so_tin"
bad = "MATCH (l:Listing) DETACH DELETE l"
assert is_read_only(ok) and not is_read_only(bad)
print("cho phép:", is_read_only(ok), "| chan:", not is_read_only(bad))
```

```
# Output
cho phép: True | chan: True
```

A.6 Recall@k, MRR và Faithfulness trên ví dụ toy

P4-08 LIVE · slide 27-28

Số toy khớp slide 27 và 28: Recall@5 = 0,67; MRR = 0,61; Faithfulness = 0,75.

```
def recall_at_k(retrieved, relevant, k):
    return len(set(retrieved[:k]) & set(relevant)) / len(relevant)

def reciprocal_rank(retrieved, relevant):
    for pos, doc in enumerate(retrieved, start=1):
        if doc in relevant:
            return 1 / pos
    return 0.0

def mrr(runs):
    return sum(reciprocal_rank(r, rel) for r, rel in runs) / len(runs)

def faithfulness(claim_supported):
    return sum(claim_supported) / len(claim_supported)

retrieved, relevant = ["d4", "d2", "d1", "d9", "d3"], {"d1", "d4", "d7"}
runs = [[["a", "x", "y"], {"a"}], [{"x", "y", "b"}, {"b"}], [{"x", "c", "y"}, {"c"}]]
assert abs(recall_at_k(retrieved, relevant, 5) - 2 / 3) < 1e-9
assert abs(mrr(runs) - (1 + 1 / 3 + 1 / 2) / 3) < 1e-9
assert faithfulness([True, True, True, False]) == 0.75
print(f"Recall@5={recall_at_k(retrieved, relevant, 5):.2f}",
      f"MRR={mrr(runs):.2f}", f"Faithfulness={faithfulness([True, True, True, False]):.2f}")

# Output
Recall@5=0.67 MRR=0.61 Faithfulness=0.75
```

Phụ lục B Tra cứu từ slide sang cell

Chữ đậm là cell LIVE. Dấu gạch ngang là slide không có cell code (chỉ có nội dung lý thuyết).

Bảng 14. Slide và cell tương ứng.

SLIDE	CELL	SLIDE	CELL
1	—	17	P3-01, P3-02
2	—	18	—
3	—	19	P3-03
4	—	20	P3-04 , P3-05, P3-06, P3-08
5	—	21	P3-09 , P3-10 , P3-11
6	P1-01, P1-02, P1-03 , P1-04, P1-05	22	P3-12
7	P1-06 , P1-07	23	P3-06, P3-07, P3-13
8	P1-08	24	P4-01
9	P1-09, P1-10, P1-11	25	P4-03 , P4-04, P4-05
10	P2-01, P2-02	26	P4-06 , P4-07
11	P2-03 , P2-04	27	P4-08
12	P2-05, P2-06	28	P4-02, P4-09
13	P2-07 , P2-08	29	P4-10, P4-12
14	P2-09 , P2-10	30	P4-10, P4-11 , P4-13
15	P2-11	31	—
16	P2-11	32	—